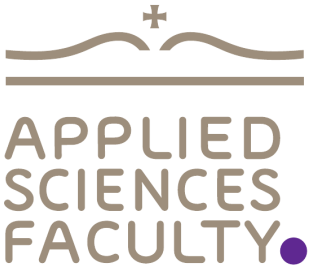
C++ — лекція 25. auto і decltype





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Виведення типів

- Із зростанням використання шаблонів, STL та розвитком шаблонного метапрограмування, вказання конкретних типів стає все складнішим. Звична ситуація: нас конкретний тип не цікавить — лише б, скажімо, створити змінну такого типу, компілятор цей тип теж знає. Або є серія змінних, які мають бути однотипними, хочеться вказати "хай всі решта будуть того ж типу". А відповідних синтаксичних засобів немає. Доводиться або писати простирадла оголошень, або вгадувати (і, місцями, помилятися). Наприклад:

```
map< vector<int>::const_iterator, list <double>::iterator >
    :: const_iterator it = mv.begin(); // [1]

template <class T, class U>
??? getMax(const T&, const U&); // [2]
```

- C++11, нарешті, надало засоби спрощення такого виведення типів — auto і decltype.

- З їх допомогою приклади вище набувають вигляду:

```
auto it = mv.begin(); // [1]

template <class T, class U>
auto getMax(const T& x, const U& y) -> decltype( x>y? x : y);
                                        // [2]
```

- Що цікаво, auto якраз у такому сенсі було в C++ ще на початку 1980-х. Однак, було викинуто, щоб уникнути конфлікту із звичкою тодішнього C припускати неявний int там, де не було вказано нічого. При чому, дана можливість вже до кінця 80-х була неактуальною і в C.

Розглянемо їх детальніше.

auto

- Ключове слово auto[1] вказує створити змінну того ж типу, що ініціалізатор.
- Прості приклади:

```
auto x = 5;              // Тип x -- int

double getPi(){ return 3.14159265358979323846; }
auto y = getPi();        // Тип y -- double

vector<int> v1 {3,4,5};
for( auto itr = v1.begin(); itr !=v1.end(); ++itr ){}
//        ^ itr --- vector<int>::iterator

vector<int> v2 {3,4,5};
vector<double> v3 {1.3,2.87,3.11};
auto ip = v2[0]*v3[0];
for(auto i = 1u; i<v2.size(); ++i){
    ip += v2[i]*v3[i];
}
```

[1] Раніше воно означало оголошення локальної змінної, але так як в такому сенсі давно не використовувалося, було використано з іншою ціллю. Насправді, створення нових ключових слів теж є нетривіальною проблемою з точки зору зворотної сумісності.

Езотеро-auto

- Однак, auto допускає і значно просунутіше використання. Є багато загадкових чи езотеричних типів, змінні яких теж би хотілося мати:

- ```
  auto f1 = [](){cout << "Hello!" << endl;};
  f1();
  auto f2 = [](int x, int y){cout << "x = " << x << ",
                                  y = " << y << endl;};
  auto f3 = bind(f2, placeholders::_1, 5);
  f3(1);
  ```

- C++14 дозволяє ще й такі фокуси:

  ```
  auto fn4 = [](const auto& x) { return 2*x; }; // C++14+ !
  ```
  ```

- Насправді, інших способів створити такі змінні, не існує. Схожого ефекту можна добитися використовуючи клас function<>, про який — окрема розмова, але отриманий код буде повільнішим.
- Під час тестової компіляції, фактичний тип f1 для gcc-5.2 (MinGW/MSYS2/x86-64) був `main()::__lambda0`, для MSVC 2015 — `main::<lambda_47afe6c57cc99f039b9314240e7f5713>`.
- Фактичний типу f3 — `std::_Bind<main()::__lambda1(std::_Placeholder<1>, int)>` і

```
std::_Binder<std::_Unforced,main::
        <lambda_b1269e1bcc5b9873db4e40be274e9f01> &,
        const std::_Ph<1> &,int>
```

- Тобто, в явному вигляді їх задавати неможливо — вони системозалежні, компіляторозалежні, навіть, може, фазо-місяцево-залежні.

decltype

- Іноді auto таки замало. Крім вже наведеного визначення типу результату функції (із використанням нового способу визначати тип результату[2]):

```
template <class T, class U>
auto getMax(const T& x, const U& y) -> decltype( x>y? x : y)
{
    return x>y? x : y;
};
```

є ще декілька важливих використань:

[2] Рекомендується використовувати такий "auto name -> type" підхід лише у випадках, коли тип результату залежить від типу аргументів, і не зловживати у інших.

- Створення контейнера, тип елементів якого виводиться:

```
vector<int> v2 {3,4,5}; vector<double> v3 {1.3,2.87,3.11};
vector< decltype( v2[0]*v3[0] ) > mv;
transform(v2.begin(), v2.end(), v3.begin(), back_inserter(mv),
                    multiplies <decltype( v2[0]*v3[0] )>());
```

- Використання typedef чи using, дозволяє зробити цей код симпатичнішим:

```
//typedef decltype( v2[0]*v3[0] ) res_type;
using res_type = decltype( v2[0]*v3[0] );
vector< res_type > mv2;
transform(v2.begin(), v2.end(), v3.begin(), back_inserter(mv),
                    multiplies <res_type>());
```

- Або контейнера із компаратором, який оголошено десь поруч:

```
struct SomeData{
    int x, y;
};
auto Comparator = [](const SomeData& a, const SomeData& b){
                                                return a.y<b.y; };
set<SomeData, decltype(Comparator)> s2(Comparator);
s2.emplace( SomeData{1,2} );
```

Детальніше про auto

- Виведення типів auto, за одним винятком, відповідає виведенню типів аргументів шаблонів функцій:

```
template<typename T>
void fn(T x);
    ..................
fn(smth);            // Вивести T із типу smth ––– того чи
                     // іншого виразу
auto x = smth;       // Вивести тип x із виразу smth
```

- Але, зауважте – тип аргументу шаблону і тип аргументу функції можуть не збігатися:

```
template<typename T>
void fn(const T& x);
    ..................
X smth;
fn(smth);            // T == X, const& відкинуто.
```

- Виняток наступний. Для такого коду:

```
auto y = {1,2,3};
```

auto вважатиме, що типу y — std::initializer_list<int>, а шаблони не вважають таку ситуацію можливою.

- auto можна використати для виведення типів і у хитріших ситуаціях:

```
int x = 0;
auto                 a = x;    // int
auto &               b = x;    // int&
auto                 c = &x;   // int*
auto *               d = &x;   // int*
const auto           e = x;    // const int
const auto &         f = x;    // const int&
const auto *         g = &x;   // const int*
const auto * const   h = &x;   // const int *const
volatile auto        i = x;    // volatile int
```

- Зверніть увагу, зірочка в `auto * d = &x;` нічого не додає до виведення типів, хіба що – якщо передати не вказівник, не скомпілюється.
- `auto&&` веде себе аналогічно шаблонів – може створювати чи lvalue reference, чи rvalue reference, в залежності від ініціалізатора.
  - ◇ Для шаблонів така поведінка називається forwarding regefence.
  - ◇ Це зручно, наприклад, в range loop.

- Як і при виведенні аргументів шаблонів, auto відкидає модифікатори змінної, яка служить ініціалізатором, такі як & чи const/volatile.
- Тому тут тип i1 буде int, а не int&.

```
int&        rx = x;
auto        i1 = rx;     // int
const int   cx = 5;
auto        i2 = cx;     // int
```

- Але:

```
int*        px  = &x;
auto        j1  = px;    // int*
const int*  pcx = &x;
auto        j2  = pcx;   // const int*
```

- Екзотичне використання:

```
auto (*p)() -> int;      // p -- вказівник на функцію
                         // без аргументів, що повертає int
```

- Створити вказівник на функцію, яка повертає тип T, який виводиться з типу p.

```
auto (*q)() -> auto = p;
q = []() { return 1;};   // Ілюстрація використання
```

- Можна оголошувати кілька змінних одним auto, але лише коли їх тип збігається:

```
auto x2=5, x3=7;          // OK
//auto y2=5, y3=7.3;      // Bad
```

- На загал, підтримуються обидва різновиди ініціалізації:

```
auto z1=5;   // int
auto z2(5);  // int
```

Але узагальнена ініціалізація, до C++17, змусить вважати тип initializer_list<>:

```
auto z3={5}; // std:: initializer_list <int>
auto z4{5};  // std:: initializer_list <int>
```

Це породжувало проблеми. (Див. главу про універсальну ініціалізацію).

- В C++17 правила змінили:

```
auto z5 = { 5, 6 };      // std:: initializer_list <int>
auto z6 = { 7, 8.0 };    // Помилка -- тип вивести не можна
auto z7{ 9, 10 };        // Помилка в C++17
auto z8 = { 11 };        // std:: initializer_list <int>
auto z9{ 12 };           // int
```

- Згідно прикладу, наведеного Саттером в главі його нової книги[3] (GotW #93, `https://herbsutter.com/2013/06/13/gotw-93-solution-auto-variables-part-2/`.), в наступній ситуації:

  ```
  gadget get_gadget();
  widget w = get_gadget();
  ```

  навіть якщо gadget конвертується у widget, може бути створено тимчасовий об'єкт, тому краще зробити:

  ```
  auto w = get_gadget();
  ```

  Але навіть якщо кінцевий тип і має бути саме widget, згідно Саттера, наступний код є більш виразним — він безпосередньо наголошує на перетворенні:

  ```
  auto w = widget{ get_gadget() };
  ```

- Плюс, така ініціалізація не дозволить відбутися неявному перетворення із втратами (типу double to int).

[3] Яка, дивлячись із 2018, судячи по всьому, ніколи не вийде.

- Ще один приклад Саттера, з того ж GotW #93. Написати так:

  SomeType x;

  дозволяє випадково забути проініціалізувати x[4]. А так змусить ініціалізувати:

  auto x = SomeType{};

- Звичний код:

  vector<**double**> v;
  int sz = v.size();

  зразу містить ймовірну проблему. Тип результату size() завжди більший за int[5]. Тому:

  vector<**double**> v;
  auto sz = v.size();

- Це — не говорячи про вирази виду "type z = z + y;".

[4] Якщо цей тип є вбудованим, має конструктор за замовчуванням, який тут не влаштовує тощо.

[5] Зазвичай це щось типу size_t, хоча формально стандарт каже, що це vector<T>::size_type.

- Але, наприклад, в оголошенні змінної циклу C++23 були проблеми:

```
for(auto i = 0; i < N; ++i) { ... } // Тип i -- int
for(auto i = 0u; i < N; ++i) { ... } // Тип i -- unsigned int,
                                     // ймовірно -- менший, ніж size_t
for(auto i = 0ull; i < N; ++i) { ... } // Тип i -- unsigned long long,
                                       // може бути завеликим
```

нарешті в C++23 з'явилися літерали для size_t:

```
for(auto i = 0uz; i < N; ++i) { ... } // Тип i -- size_t
```

- Важливий виняток — критичний до продуктивності код, де auto вибере завеликий для даної ситуації тип (наприклад, в масиві розміром 16 елементів, достатньо int, і він вдвічі менший за size_t на x86-64). Хоча і тут краще викрутитися способом, наведеним вище — явним вказанням типу. Зокрема, див. std::as_signed() і std::as_unsigned().

За більш детальним аналізом див.

- `https://herbsutter.com/2013/06/07/gotw-92-solution-auto-variables-part-1/` — GotW #92 Solution: Auto Variables, Part 1.
- `https://herbsutter.com/2013/06/13/gotw-93-solution-auto-variables-part-2/` — GotW #93 Solution: Auto Variables, Part 2.
- `https://herbsutter.com/2013/08/12/gotw-94-solution-aaa-style-almost-always-auto/` — GotW #94 Solution: AAA Style (Almost Always Auto).

C++14 auto

- C++14 Додав ряд нових використань auto.
- Виведення результату функції із типу виразу в return:

```
auto f1(){
    return 5;
}
```

- Якщо return декілька — типи повинні збігатися!

```
auto f2(int x){
    if (x > 5)
        return 5;
    else
        return 7; // 7.1 would be error!
}
```

- Можливі і всілякі такі фокуси:

```
double global = 3.14;
const auto& f3() { return global; }
```

decltype(auto) – 1

- decltype(auto) вказує, що результат має бути таким, як вказав би decltype, якби його використали до аргументу return:

```
decltype(auto) f4() {return   global;}
                    // ^ int   – decltype( global )
decltype(auto) f5() {return (global);}
                    // ^ int& – decltype((global))
```

- Обидва ці засоби фіксують і випадок, коли функція нічого не повертає:

```
auto f6() {}                    // returns void
auto f7() { return f6(); } // returns void
decltype(auto) f8() { }     // returns void
```

- Важливою рисою decltype(auto) (на відміну від auto) є те, що він зберігає "referencencess" аргументу. Для звичайних змінних і більшості функцій це аж ніяк не бажано.
- Призначений він для всіляких forwarding-функцій. Не використовуйте його за межами такого контексту.

decltype(auto) – 2

- Приклад такої forwarding функції:

```
template<typename Fn, typename... Args>
decltype(auto) apply_fn(Fn fn, Args&&... args) {
    return fn(std::forward<Args>(args)...);
}
```

- Інше використання – виведення типів у рекурсивних шаблонах:

```
template<int i> struct Int {};

constexpr auto iter(Int<0>) -> Int<0>;

template<int i>
constexpr auto iter(Int<i>) -> decltype(auto) {
  return iter(Int<i-1>{});
}

int main(){
  decltype(iter(Int<10>{})) a;
}
```

Приклад взято тут:
https://stackoverflow.com/questions/24036357/

decltype(auto) – 3

- Приклади особливості функціювання:

```
int  i;
int&& f();
auto x3a = i;                    // decltype(x3a) is int
decltype(auto) x3d = i;          // decltype(x3d) is int
auto x4a = (i);                  // decltype(x4a) is int
decltype(auto) x4d = (i);        // decltype(x4d) is int&
auto x5a = f();                  // decltype(x5a) is int
decltype(auto) x5d = f();        // decltype(x5d) is int&&
auto x6a = { 1, 2 };             // decltype(x6a) is
                                 // std::initializer_list<int>
decltype(auto) x6d = { 1, 2 };   // error, { 1, 2 } is not an expression
auto *x7a = &i;                  // decltype(x7a) is int*
decltype(auto)*x7d = &i;         // error, declared type is not plain
                                 // decltype(auto)
```

- Взято тут: `https://stackoverflow.com/a/24109800`, там, у свою чергу із dcl.spec.auto.

- Можливе рекурсивне використання auto:

```
auto factorial (unsigned x){
    if (x==1 || x==0){   return 1u;                    }
    else{                return factorial (x-1) * x;   }
}
```

  Суфікс u в 1u — важливий! Без нього тип виведеться int і конфліктуватиме із наступним return.

- Використання в лямба-функціях вже згадувалося раніше:

```
auto fn4 = [](const auto& x) { return 2*x; }; // C++14+ !
```

- initializer_list<> так повертати не можна — код із `return {1,2,3};` та auto не компілюватиметься.
- Віртуальні функції не можна оголошувати з допомогою такого auto.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/auto` та `http://en.cppreference.com/w/cpp/language/function` — зокрема, про взаємодію із шаблонами.

Auto для членів класу

- Не статичні члени даних класу не можна оголошувати auto[6]:

```
class SomeType{
        // auto x = 5; // Bad
};
```

- Однак, статичні члени так оголошувати можна:

```
class SomeType{
        static inline auto x = 5; // OK
};
```

- Ремарка – в статті у виносці є ідея, як оголошувати не-статичні члени auto, якщо не можна, але дуже хочеться:

```
struct SomeType {
    decltype(42) x = 42;
};
// OR
#define AUTO(var, expr) decltype(expr) var = (expr)
struct SomeType {
    AUTO(x, 42);
};
```

[6] Дискусія на тему: https://cor3ntin.github.io/posts/auto_nsdmi/ та стара пропозиція: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2008/n2713.html

auto(x) та auto{x} – C++23

- Іноді нам потрібно передати копію функції, яка приймає константне посилання.
  - Наприклад, вихідний об'єкт не проживе достатньо довго: через паралельні обчислення, або коли видаляємо з контейнера, за допомогою std::remove() елементи, рівні елементу з цього ж контейнера.
  - Інший приклад – потреба мати кілька різних модифікацій того ж об'єкту.
- Наприклад (взято з документу, де цей засіб запропоновано), так робити – помилка, через інвалідацію:

```
void pop_front_alike(Container auto& x) {
    std::erase(x.begin(), x.end(), x.front()); // Помилка!
}
```

Її можна виправити:

```
void pop_front_alike(Container auto& x) {
    auto a = x.front();
    std::erase(x.begin(), x.end(), a);
}
```

однак, така явна копія – неочевидна.

  - Тут краще пасувало б prvalue. До C++23 його можна отримати якось так:

```
void pop_front_alike(Container auto& x) {
    using T = std::decay_t<decltype(x.front())>;
    std::erase(x.begin(), x.end(), T(x.front()));
}
```

Це доволі багатослівно.

- C++23 дозволяє записати попередню конструкцію так:

```
void pop_front_alike(Container auto& x) {
    std::erase(x.begin(), x.end(), auto(x.front()));
}
```

◇ Ще один приклад з оригінального документу:

```
class cmdline_parser {
        ..................................
    auto add_option(char const*, char const*) & -> cmdline_parser &;
    auto add_option(char const*, char const*) && -> cmdline_parser &&;
};


        ..........................................
    auto cmdline = cmdline_parser("driver");

    cmdline.add_option("-h", "show help messages")
           .add_option("-v", "show version");

    auto internal = auto(cmdline)
                        .add_option("--logging-level", "set logging level")
                        .add_option("--dump-full", "do not minimize dump");
     internal.parse(argc, argv);
```

- Формально кажучи, auto(x) та auto{x} повертають копію вихідного значення, яка є prvalue.
- Детальніше див. оригінальну пропозицію та, скажімо, "How can I force a copy of a C++ value?".

Посилання

- ”Auto Auto Auto” – курйозний приклад, auto auto auto:

```
struct A{};
using T = auto() -> auto(*)() -> auto(*)() -> A;
```

- ”‘auto{x} != auto(x)“”.

Особливості decltype

- Аргументи decltype не обчислюються! Це просто запит до компілятора під час компіляції.
- Використовувати його замість auto синтаксично можливо, але безглуздо:

```
decltype(x) k1;
decltype(x) k2 = x;
```

- Так як (val) є rvalue, decltype((val)) створює змінну не типу val, а типу посилання на val! Тобто, код:

```
double d1 = 3.14;
decltype((d1)) d2 = d1;
d2 = 5.67;
cout << "d1 = " << d1 << ", d2 = " << d2 << endl;
```

Виведе:

```
d1 = 5.67, d2 = 5.67
```

- Цитата із стандарту до попереднього прикладу[7]:
  - if e is an unparenthesized id-expression or a class member access (5.2.5), decltype(e) is the type of the entity named by e.
  - if e is an lvalue, decltype(e) is T&, where T is the type of e;
- Див. також запитання "What does decltype((...)) with double parentheses mean?" та есей "Hidden reinterpret_casts" щодо можливих небезпек.
- Основні використання decltype() стосуються написання шаблонів.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/decltype`.

[7] А тут є приклад проблем, що можуть виникнути через це, щоправда, якщо писати достатньо збочений код: `https://quuxplusone.github.io/blog/2020/01/22/expression-list-in-functional-cast/`.

Типи результатів функції

Функція	14-20[8]	P2266[9]
`auto a(T x) -> decltype( x ) { return x; }`	T	T
`auto b(T x) -> decltype((x)) { return (x); }`	T&	T&
`auto c(T x) -> decltype(auto) { return x; }`	T	T
`auto d(T x) -> decltype(auto) { return (x); }`	T&	T&&
`auto e(T&& x) -> decltype( x ) { return x; }`	ill-f.	T&&
`auto f(T&& x) -> decltype((x)) { return (x); }`	T&	ill-f.
`auto g(T&& x) -> decltype(auto) { return x; }`	ill-f.	T&&
`auto h(T&& x) -> decltype(auto) { return (x); }`	T&	T&&

- Підсумок із `https://quuxplusone.github.io/blog/2021/05/04/p2266-decltype-auto/`:
- ill-f. – ill-formed, не має компілюватися.

[8] C++14, C++17, C++20.

[9] P2266 – пропозиція до C++23.

std::declval()

- std::declval() дозволяє використовувати в ролі аргументу decltype() члени класів без конструкторів.
- Технічно це функція. Однак, вона — чисто службова функція часу компіляції, викликати, брати адресу, і т.д, не можна!

```
template<class T>
typename std::add_rvalue_reference<T>::type declval();
```

Знаходиться в <utility>.

- Приклад:

```
class WithoutDefaultConstructors {
public:
    WithoutDefaultConstructors(const WithoutDefaultConstructors&) { }
    int fn() const { return 5; }
};

decltype(std :: declval<WithoutDefaultConstructors>().fn()) w;
```

- Детальніше див. http://en.cppreference.com/w/cpp/utility/declval.

std::invoke_result – C++17, заміна для std::result_of

- Клас std::invoke_result із <type_traits> дозволяє вказати тип, рівний типу, який повертається викликом певної функції чи іншого callable:

```
std :: invoke_result< decltype(&getPi) >::type    vd1 = 5.55; // double

auto f4 = [](){ return 123; };
std :: invoke_result< decltype(f4) >::type        vi2 = 7; // int

auto f5 = [](double c){ return c-1; };
std :: invoke_result< decltype(f5), double >::type vd2 = 7.54; // double

struct Functor1 {
    double operator()(int, int) { return 3.14; };
    int operator()(int) { return 7;}
    short mfn(char) { return 0; }
};

std :: invoke_result<Functor1, int, int>::type      vd3 = 3.14; // double
std :: invoke_result<Functor1, int>::type           vi = 14;    // int
std :: invoke_result< decltype(&Functor1::mfn), Functor1, char>::type
                                                    vs = 5; // short
```

std::result_of – застаріла в C++17, викинута в C++20

- Замінена на std::invoke_result.
- Подробиці: `https://en.cppreference.com/w/cpp/types/result_of#Notes`.
- Клас std::result_of із <type_traits> дозволяє вказати тип, рівний типу, який повертається викликом певної функції чи іншого callable:

```
double getPi() {      return 3.14159265358979323846;     }

std :: result_of< decltype(&getPi)() >::type    vd1 = 5.55; // double

auto f4 = [](){ return 123; };
std :: result_of< decltype(f4)() >::type        vi2 = 7; // int

auto f5 = [](double c){ return c-1; };
std :: result_of< decltype(f5)(double) >::type vd2 = 7.54; // double
```

- Не працює із перевантаженими функціями — вибір перевантаження із вказівниками не працює.
- Про <type_traits> буде окрема розмова.

std::result_of – застаріла в C++17, викинута в C++20

- Однак, із перевантаженими членами класів проблем немає:

```
struct Functor1 {
    double operator()(int, int) { return 3.14; };
    int operator()(int) { return 7;}
    short mfn(char) { return 0; }
};

std :: result_of<Functor1(int, int)>::type      vd3 = 3.14; // double
std :: result_of<Functor1(int)>::type           vi  = 14;   // int
```

Як і з вказівниками на члени:

```
std :: result_of< decltype(&Functor1::mfn)(Functor1, char)>::type
                                                vs  = 5; // short
```

Хоча, безперечно, синтаксис той ще.

std::result_of – застаріла в C++17, викинута в C++20

- В C++14 можливі навіть такі фокуси:

```
#if __cplusplus >= 201402L
auto fn1(){
    return 5;
}
#endif // __cplusplus
    ..............

#if __cplusplus >= 201402L
    std :: result_of< decltype(&fn1)() >::type      vi0 = 11;   // int
#endif // __cplusplus
```

- Детальніше див. `http://en.cppreference.com/w/cpp/types/result_of`. Зокрема, в C++14 він трішки толерантніший — якщо аргумент не є callable, просто не міститиме type. В C++11 така ситуація вважатиметься помилкою.
- Про різницю між decltype та result_of, див. `http://stackoverflow.com/a/35834192`. Якщо коротко, в деяких ситуаціях (із шаблонами із змінною кількістю аргументів, result_of лаконічніший, крім того, його можна застосовувати до таких речей, до яких decltype напряму не застосуєш, наприклад, до вказівників на методи).

std::common_type

- **template**< **class**... T >
  struct common_type;

  Використовується для визначення типу, сумісного із всіма перерахованими.
- Якщо такий існує, вона міститиме його синонім із іменем type, інакше не міститиме такого типу.
- C++14 додав:

  template< **class**... T >
  using common_type_t = **typename** common_type<T...>::type;
- Приклад:

```
#include <type_traits>

template <class T>
struct Number { T n; };

template <class T, class U>
Number<typename std::common_type<T, U>::type> operator+(
                                                const Number<T>& lhs,
                                                const Number<U>& rhs) {
    return {lhs.n + rhs.n};
}
```



